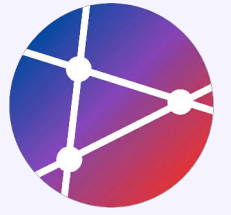


WEEK 1

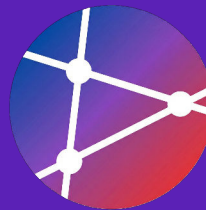
AI OPEN SOURCE SOFTWARE



1. Metrics That Matter

Prof. Sejin Chun

 Dong-A University



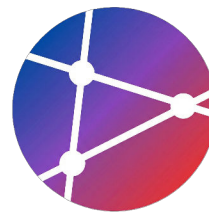
INTRODUCTION

중요한 소프트웨어 개발 메트릭

본 강의의 전반적인 주제를 소개하고
측정의 중요성을 이해합니다.



학습 목표



핵심 메트릭 이해

소프트웨어 전달 성능을 측정하는 핵심 메트릭의 개념과 필요성을 명확히 이해합니다.



DORA Metrics 적용

DORA의 4가지 핵심 지표(Lead Time, Frequency, MTTR, Failure Rate)를 실무에 적용합니다.



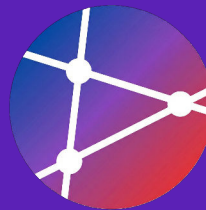
Flow Metrics 활용

Flow Metrics(Cycle Time, Throughput 등)를 활용하여 개발 프로세스 흐름을 최적화합니다.



데이터 기반 개선

수집된 메트릭 데이터를 바탕으로 의사결정을 내리고 지속적인 프로세스 개선을 수행합니다.

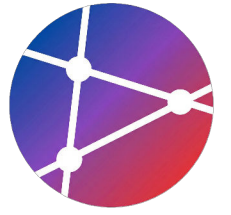


MOTIVATION

왜 메트릭이 중요한가?

소프트웨어 개발 프로세스에서
측정의 필요성과 핵심 가치를 탐구합니다.



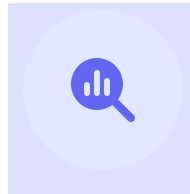


“ 측정할 수 없으면 개선할 수 없다 ”



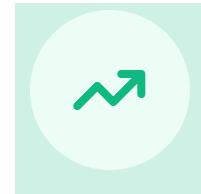
측정 없음

현상태 파악 불가



측정 시작

개선 포인트 발견

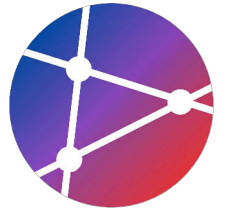


지속 측정

성과 검증 및 최적화



메트릭의 가치



가시성 (Visibility)

현재 상태를 객관적으로 파악하여 모호함을 제거하고, 전체 프로세스의 투명성을 확보합니다.



예측성 (Predictability)

과거 데이터를 기반으로 미래 성과를 예측하고, 데이터에 기반한 장기 계획을 수립합니다.



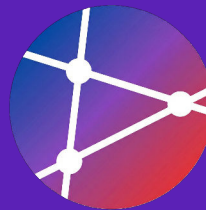
책임성 (Accountability)

명확한 팀 목표를 설정하고, 정량적 지표를 통해 성과를 공정하고 투명하게 평가합니다.



개선성 (Improvability)

병목 지점을 정확히 파악하여 제거하고, 지속적인 최적화를 통해 프로세스 효율을 높입니다.



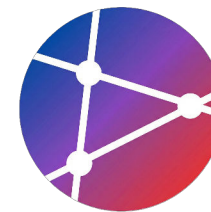
SECTION 03

DORA Metrics: 4가지 핵심 지표

Google의 DevOps Research and Assessment(DORA) 팀이 수년간의 연구를 통해 발견한 소프트웨어 전달 성능의 핵심 지표입니다.



DORA Metrics 개요



Google의 DevOps Research and Assessment (DORA) 팀이 6년 이상의 연구를 통해 발견한 **소프트웨어 전달 성능(Software Delivery Performance)**의 핵심 지표입니다.



Lead Time for Changes

SPEED

변경 리드 타임

코드 커밋부터 코드가 **프로덕션** 환경에 성공적으로 배포되기까지 걸리는 시간입니다.



Deployment Frequency

SPEED

배포 빈도

성공적인 소프트웨어를 **프로덕션** 환경에 배포하는 빈도입니다. (예: 일일 배포 횟수)



Mean Time to Recovery

STABILITY

평균 복구 시간 (MTTR)

프로덕션 환경에서 서비스 중단이나 장애가 발생했을 때, 이를 복구하는 데 걸리는 평균 시간입니다.



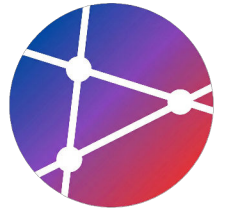
Change Failure Rate

STABILITY

변경 실패율

프로덕션 배포 중 실패하거나 핫픽스, 롤백 등의 즉각적인 조치가 필요한 변경의 비율입니다.

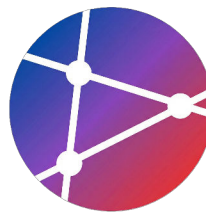
Lead Time for Changes



DEFINITION

코드 커밋(Commit) 시점부터 코드가 성공적으로 프로덕션 환경에 배포(Deploy)될 때까지 걸리는 총 소요 시간

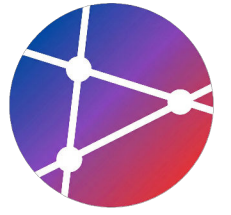




Lead Time for Changes: 성과 수준

코드 커밋 시점부터 프로덕션 배포가 완료될 때까지 소요되는 시간을 기준으로 성과 수준을 4단계로 분류합니다.

PERFORMANCE LEVEL	LEAD TIME	의미 및 특징
 Elite	 1시간 미만	진정한 지속적 배포(Continuous Deployment) 달성. 커밋 즉시 배포되는 수준.
 High	 1일 ~ 1주	매우 빠른 피드백 사이클. 주 단위 스프린트보다 빠르게 배포 가능.
 Medium	 1주 ~ 1개월	정기적 배포. 일반적인 스크럼/스프린트 주기에 맞춘 배포 속도.
 Low	 1개월 이상	느린 피드백 사이클. 릴리즈 주기가 길어 변경 사항에 대한 리스크가 높음.



Lead Time for Changes: 개선 방법



작은 배치 크기 (Small Batch Size)

작업 단위를 작게 나누어 복잡도와 리스크를 줄이고 피드백 속도를 높입니다.



자동화된 테스트 및 배포

CI/CD 파이프라인을 구축하여 빌드부터 배포까지의 과정을 자동화합니다.



Trunk-Based Development

긴 수명의 브랜치를 피하고 메인 트렁크에 빈번하게 코드를 통합합니다.



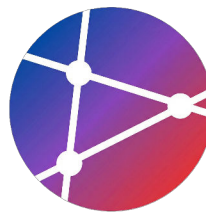
Feature Flags 활용

배포(Deploy)와 출시(Release)를 분리하여 준비된 기능만 안전하게 노출합니다.



코드 리뷰 프로세스 간소화

리뷰 절차를 효율화하여 대기 시간을 최소화하고, 동료 간의 빠른 피드백 루프를 형성합니다.



</> Lead Time 측정 자동화

GitHub Actions 활용

코드 커밋부터 병합까지의 시간을 자동으로 계산하여 **Lead Time for Changes** 지표를 추적할 수 있습니다.

i 작동 원리

- Pull Request가 **Merged**(병합)될 때 트리거
- 첫 커밋 시간(First Commit)과 병합 시간(Merge Time) 추출
- 두 시간의 차이를 초 단위로 계산하여 출력

📄 파일명: .github/workflows/metrics.yml

```
metrics.yml

# .github/workflows/metrics.yml
name: Track Lead Time
on:
  pull_request:
    types: [closed]

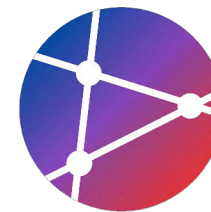
jobs:
  track-lead-time:
    # PR이 병합되었을 때만 실행
    if: github.event.pull_request.merged == true
    runs-on: ubuntu-latest
    steps:
      - name: Calculate Lead Time
        run: |
          FIRST_COMMIT_TIME="${{ github.event.pull_request.created_at }}"
          MERGE_TIME="${{ github.event.pull_request.merged_at }}"

          # 시간 차이 계산 (초 단위)
          LEAD_TIME=$((date -d "$MERGE_TIME" +%s) - $(date -d "$FIRST_COMMIT_TIME" +%s))

          echo "Lead Time: $LEAD_TIME seconds"
```



Deployment Frequency



정의 (Definition)

조직이 성공적으로 **프로덕션 환경**에 코드를 배포하는 빈도



높은 배포 빈도

High Deployment Frequency



작은 변경 단위

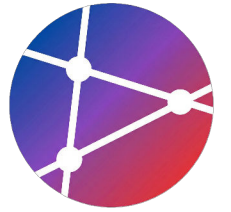
Small Batch Size



낮은 리스크

Low Risk & Easy Rollback

Deployment Frequency: 성과 수준

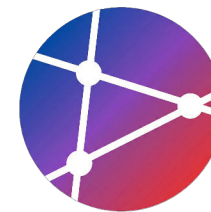


조직이 성공적으로 프로덕션 환경에 릴리스를 배포하는 빈도를 기준으로 성과를 측정합니다.

PERFORMANCE LEVEL	DEPLOYMENT FREQUENCY	의미 및 특징
 Elite	 하루에 여러 번	On-demand 배포. 비즈니스 요구사항에 따라 필요할 때 즉시 배포 가능한 상태.
 High	 주 1회 ~ 일 1회	자주 배포(Frequent Deployment). 지속적 전달(CD)이 안정적으로 정착된 단계.
 Medium	 월 1회 ~ 주 1회	정기 배포. 스프린트 종료 등 일정 주기에 맞춰 배포가 이루어지는 단계.
 Low	 월 1회 미만	드문 배포. 배포 자체가 큰 이벤트가 되며 실패 리스크가 상대적으로 높음.



Deployment Frequency: 개선 방법



CI/CD 파이프라인 자동화

빌드, 테스트, 배포 과정을 완전히 자동화하여 수동 개입을 없애고 배포 속도를 높입니다.



배포 프로세스 표준화

모든 마이크로서비스와 환경에 동일한 배포 절차를 적용하여 예측 가능성을 확보합니다.



Blue-Green / Canary 배포

무중단 배포 전략과 점진적 트래픽 전환을 통해 배포 리스크를 최소화합니다.



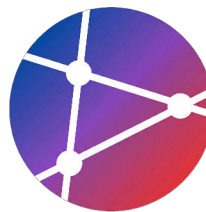
자동화된 롤백 메커니즘

배포 후 장애 감지 시, 사람의 개입 없이 즉시 안정된 이전 버전으로 복구하는 시스템을 구축합니다.



승인 프로세스 간소화

복잡한 결재 단계를 줄이고, 동료 리뷰(Peer Review)와 자동화된 품질 게이트로 승인 과정을 대체합니다.



Deployment Frequency 측정

배포 빈도 추적 자동화

GitHub Actions의 **deployment** 이벤트를 활용하여 프로덕션 배포 횟수와 환경 정보를 자동으로 기록합니다.

🎯 핵심 포인트

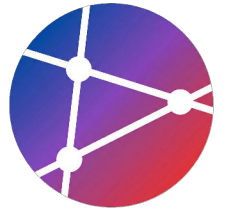
- **Deployment 이벤트:** 배포 생성 시 자동으로 트리거됨
- **실시간 로깅:** 배포 시점(Date)과 대상 환경(Environment) 기록
- **데이터 활용:** 추후 주간/월간 배포 횟수 집계에 활용 가능

 워크플로우 예시

```
deploy-tracker.yml

# GitHub Actions로 배포 빈도 추적
name: Track Deployments
on:
  deployment:

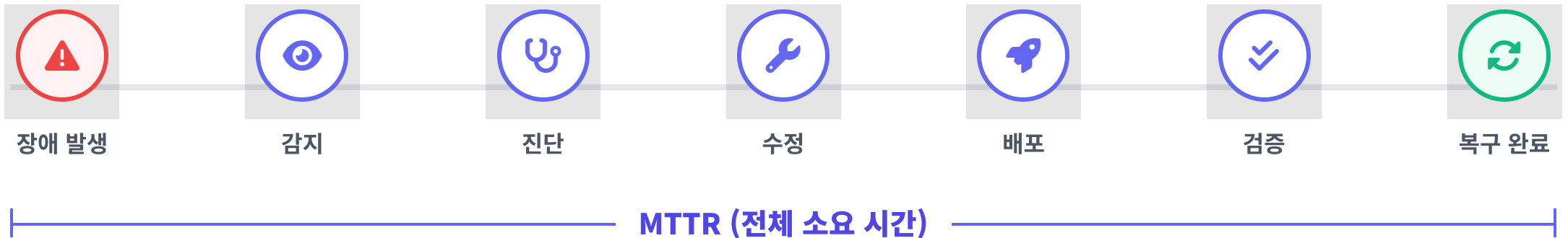
jobs:
  track-deployment:
    runs-on: ubuntu-latest
    steps:
      - name: Log Deployment
        run: |
          echo "Deployment at: $(date)"
          echo "Environment: ${ github.event.deployment.environment }"
```



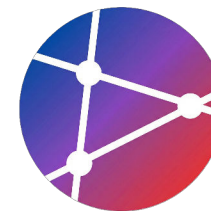
MTTR: 정의

Mean Time to Recovery (평균 복구 시간)

서비스 중단(Incident)이 발생한 시점부터 시스템이 정상 상태로 완전히 복구될 때까지 걸리는 평균 시간을 의미합니다.



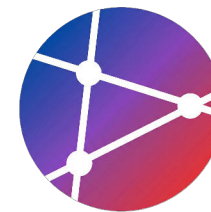
MTTR: 성과 수준



서비스 중단 발생 시 정상 상태로 복구하는 데 걸리는 평균 시간(Mean Time to Recovery)을 기준으로 성과 수준을 4단계로 분류하여 관리합니다.

PERFORMANCE LEVEL	MTTR CRITERIA	의미 및 특징
 Elite	 1시간 미만	즉각 복구 - 장애 감지 및 복구가 고도로 자동화되어 사용자 영향을 최소화함.
 High	 1시간 ~ 1일	빠른 복구 - 체계적인 대응 프로세스와 온콜 시스템으로 하루 이내 서비스 정상화.
 Medium	 1일 ~ 1주	정상 복구 - 수동 개입이 필요하며 복잡한 문제 해결을 위해 수일이 소요될 수 있음.
 Low	 1주 이상	느린 복구 - 장애 원인 파악 및 수정에 장시간 소요되며 서비스 안정성이 낮음.

MTTR 개선 방법



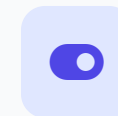
모니터링 및 알림

시스템 이상 징후를 실시간으로 감지하고
담당자에게 즉시 전파하는 관제 시스템 구축



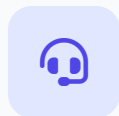
자동화된 롤백

배포 후 치명적 오류 감지 시 사람의 개입 없이 이전
안정 버전으로 즉시 복구



Feature Flags

코드 배포와 기능 출시를 분리하여, 문제 발생 시
해당 기능만 즉시 비활성화(Off)



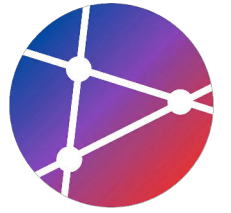
온콜(On-call) 프로세스

명확한 장애 대응 당번제 및 에스컬레이션 경로를
수립하여 대응 지연 최소화



포스트모템 문화

장애 발생 후 비난 없는 회고(Blameless Post-mortem)를 통해 근본 원인 파악 및 재발 방지



MTTR: 모니터링 예시

Service Health Check

MTTR(평균 복구 시간)을 단축하기 위해서는 장애 발생을 **즉시 감지**하는 것이 필수적입니다. GitHub Actions를 사용하여 주기적인 헬스 체크를 자동화할 수 있습니다.

구성 요소

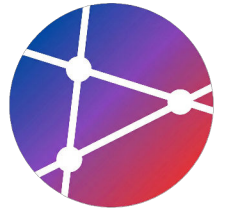
- **Schedule:** 5분 간격(* / 5) 실행
- **Curl:** HTTP 상태 코드 확인 (-f 옵션)
- **Alert:** 실패 시 알림 전송 로직 수행

 파일명: .github/workflows/health-check.yml

health-check.yml

```
# GitHub Actions로 헬스체크 및 알림
name: Health Check
on:
  schedule:
    - cron: '* / 5 * * * *' # 5분마다 실행

jobs:
  health-check:
    runs-on: ubuntu-latest
    steps:
      - name: Check Service
        run: |
          # 서비스 접속 시도, 실패 시 오류 처리
          if ! curl -f https://myapp.com/health; then
            echo "Service is down!"
```



Change Failure Rate

정의 (Definition)

프로덕션 배포 중 **실패하거나 즉각적인 롤백/수정**이 필요한 변경 사항의 비율을 의미합니다.

단순한 배포 실패뿐만 아니라, 배포 후 서비스 장애를 유발하여 긴급 패치가 필요한 경우도 포함됩니다.



안정성의 척도

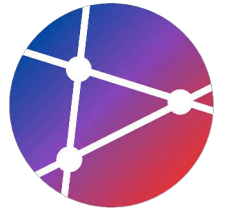
낮은 실패율은 배포 프로세스의 신뢰성과 품질 보증 단계가 효과적임을 나타내는 핵심 지표입니다.

CALCULATION FORMULA

$$\frac{\text{실패한 배포 수}}{\text{전체 배포 수}} \times 100 \%$$

CALCULATION EXAMPLE

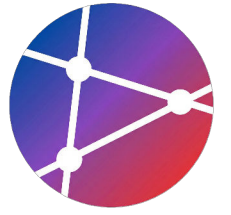
배포 10회 → 실패 1회 = 10%



Change Failure Rate: 성과 수준

프로덕션 배포 중 실패하거나 롤백이 필요한 비율을 기준으로 시스템의 안정성을 4단계로 분류합니다.

PERFORMANCE LEVEL	FAILURE RATE	의미 및 특징
 Elite	 0% - 15%	매우 안정적. 배포 실패가 거의 없으며, 높은 신뢰성을 보장함.
 High	 16% - 30%	안정적. 관리 가능한 수준의 실패율이며, 빠른 복구가 가능함.
 Medium	 31% - 45%	보통. 배포 실패가 종종 발생하며, 품질 관리 개선이 필요함.
 Low	 46% 이상	불안정. 잦은 배포 실패로 인해 서비스 신뢰도가 낮고 리스크가 높음.



Change Failure Rate: 개선 방법

배포전 테스트



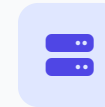
포괄적인 자동화 테스트

단위, 통합, E2E 테스트 자동화로 결함 조기 발견



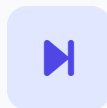
코드 리뷰 강화

엄격한 리뷰 프로세스로 코드 품질 및 안정성 확보



스테이징 환경 활용

프로덕션과 유사한 환경에서 배포 전 사전 검증



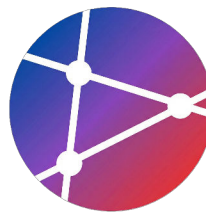
Progressive Delivery

점진적 배포(Canary 등)로 실패 영향 범위 최소화



자동화된 품질 게이트

배포 파이프라인 내 품질 기준 미달 시 배포 차단



Change Failure Rate 추적

배포 결과 자동 집계

프로덕션 배포의 성공 및 실패 여부를 자동으로 감지하여 **Change Failure Rate**(변경 실패율) 계산을 위한 기초 데이터를 수집합니다.

모니터링 포인트

- `deployment_status` 이벤트 활용
- 배포 상태(success/failure) 실시간 확인
- 실패 시 알림 전송 또는 메트릭 저장소 기록

 워크플로우 예시

track-failure.yml

```
# 배포 성공/실패 결과 추적
name: Track Deployment Result
on:
  deployment_status:

jobs:
  track-result:
    runs-on: ubuntu-latest
    steps:
      - name: Log Result
        run: |
          STATUS="${{ github.event.deployment_status.state }}"
          # 상태에 따른 분기 처리
          if [ "$STATUS" = "success" ]; then
            echo "✅ Deployment succeeded"
```



SECTION 03

Flow Metrics

작업 흐름 측정

DORA Metrics를 보완하여
개발 프로세스의 효율성을 측정하는
핵심 지표들을 알아봅니다.

The logo for Flow Metrics, featuring a stylized blue wave icon to the left of the text "Flow Metrics" in a bold, black, sans-serif font. A yellow swoosh underline is positioned beneath the text.

Flow Metrics 개요

DORA Metrics가 '결과'를 측정한다면, Flow Metrics는 가치가 전달되는 '**과정 (Workflow)**'의 효율성을 진단하고 최적화하는 핵심 지표입니다.



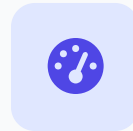
DORA 상호보완

DORA 지표만으로는 파악하기 힘든 프로세스 내부의 구체적인 병목 지점(Bottlenecks)을 식별하여 전체 가시성을 완성합니다.



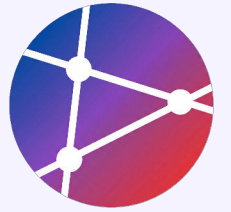
프로세스 가시성

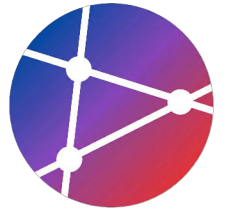
아이디어 구상부터 고객에게 가치가 전달되기까지의 전체 작업 흐름을 시각화하여 투명성을 확보합니다.



효율성 최적화

불필요한 대기 시간, 과도한 WIP(진행 중 업무), 중복 작업을 제거하여 개발팀의 생산성과 몰입도를 높입니다.





Cycle Time: 정의

Definition

작업이 실제로 시작된 시점(In Progress)부터 완료되어
사용자에게 전달될 때(Done)까지 걸리는 시간



작업 시작

In Progress



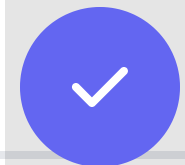
개발

Development



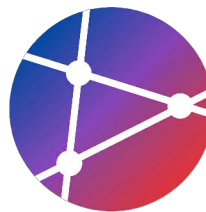
리뷰

Code Review



완료

Done



Cycle Time 측정 방법

GitHub API 활용

작업이 실제로 시작된 시점부터 완료될 때까지의 시간을 측정하기 위해 GitHub Issue의 이벤트 로그를 분석합니다.

계산 로직

- **Start Time:** 이슈에 "In Progress" 라벨이 붙은 시점을 추출
- **End Time:** 이슈가 Closed(완료)된 시점
- **Cycle Time:** (End Time - Start Time)을 시간 단위로 변환

🔗 Language: Python 3.9+

```
calculate_metrics.py

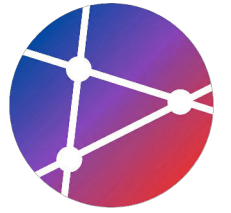
import requests
from datetime import datetime

def calculate_cycle_time(issue_number):
    # Issue 정보 가져오기
    url = f"https://api.github.com/repos/owner/repo/issues/{issue_number}"
    issue = requests.get(url)
    data = issue.json()

    # "In Progress" 라벨 추가 시간 찾기
    start_time = None
    for event in data['events']:
        if event['label']['name'] == 'In Progress':
            start_time = event['created_at']
            break
```

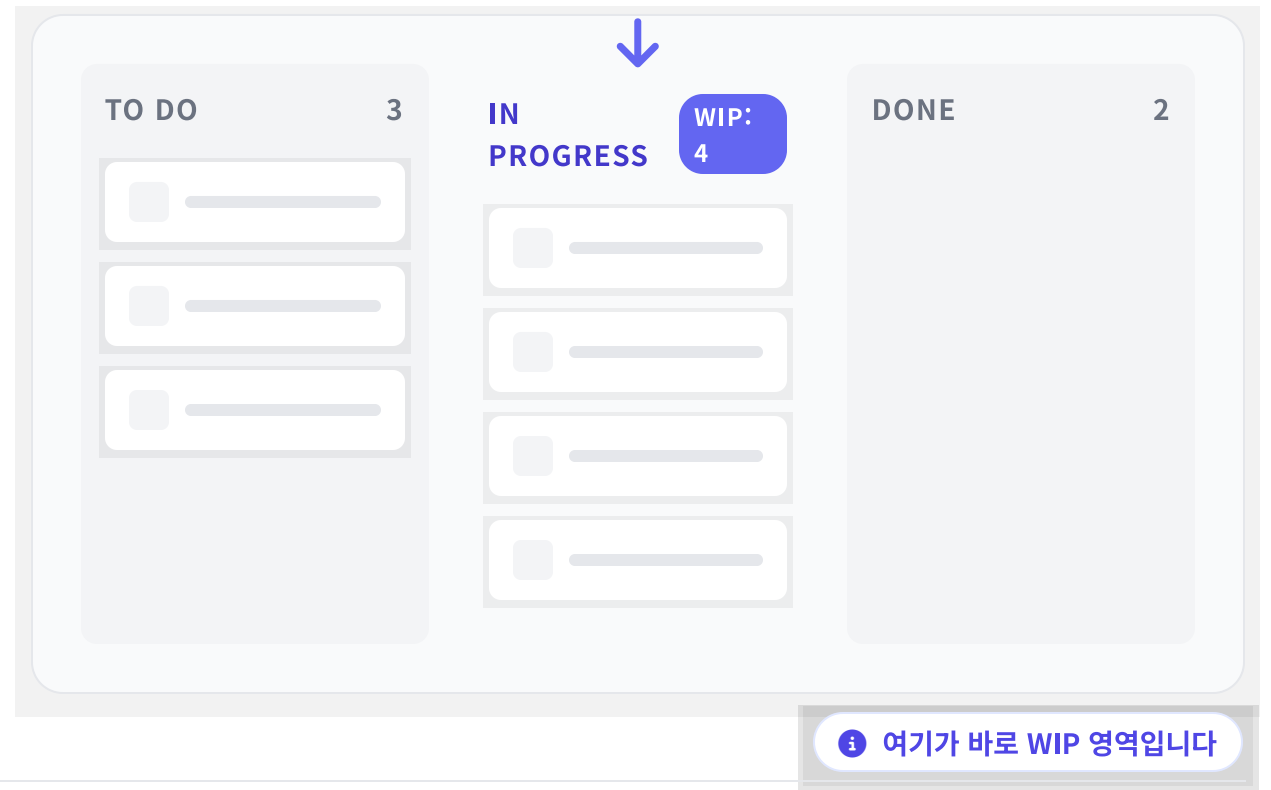


Work In Progress (WIP)

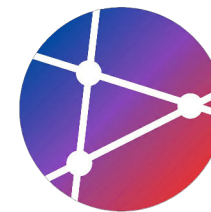


"동시에 진행 중이지만
아직 완료되지 않은 작업의 수"

- ✓ **작업 흐름의 지표:** 현재 팀이 동시에 처리하고 있는 업무량
- ✓ **병목 구간 식별:** WIP가 쌓이는 단계가 곧 프로세스의 병목
- ✓ **속도와의 관계:** WIP를 낮추면 Lead Time이 개선됨 (Little's Law)
- ⚠ **주의사항:** 과도한 WIP는 컨텍스트 스위칭 비용 증가



Little's Law (리틀의 법칙)



$$\text{Cycle Time} = \frac{\text{WIP}}{\text{Throughput}}$$


📖 변수 정의

Cycle Time: 작업 시작부터 완료까지의 시간

WIP: 동시에 진행 중인 작업의 수

Throughput: 단위 시간당 완료되는 작업 수

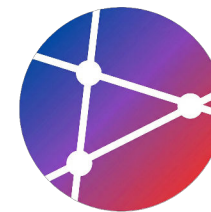
💡 핵심 인사이트

"처리량(Throughput)이 일정할 때,"

WIP → Cycle Time
↓ 감소 ↓ 단축

진행 중인 작업을 줄이면, 각 작업의 완료 속도가 빨라집니다.

WIP 제한의 이점



집중도 향상

한 번에 너무 많은 일을 처리하려 하지 않고, 현재 진행 중인 작업에 온전히 집중하여 업무 효율과 품질을 높입니다.



빠른 완료

대기 시간이 줄어들고 전체적인 사이클 타임(Cycle Time)이 단축되어, 기능의 빠른 배포와 피드백이 가능해집니다.



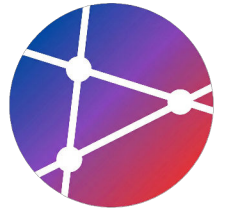
컨텍스트 스위칭 감소

잡은 업무 전환으로 발생하는 인지 부하와 시간 낭비를 최소화하여 개발자의 생산성을 유지합니다.



병목 지점 발견

작업이 쌓이는 구간이 시각적으로 명확해지므로, 프로세스상의 병목 현상을 빠르게 식별하고 해결할 수 있습니다.



GitHub Projects에서 WIP 제한

Kanban 보드 규칙 설정

WIP(Work In Progress) 제한은 동시에 진행되는 작업의 수를 물리적으로 제한하여 멀티태스킹을 방지하고 작업 흐름을 개선하는 핵심 기법입니다.

✓ WIP 제한의 효과

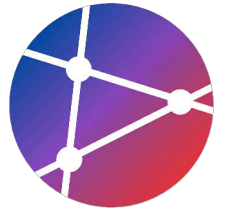
- 컨텍스트 스위칭 비용 최소화 (집중도 향상)
- 병목 구간(Bottleneck)의 즉각적인 시각화
- 리드 타임(Lead Time) 단축 및 예측 가능성 증대

📄 GitHub Projects 설명이나 규칙 문서에 명시





Throughput (처리량)

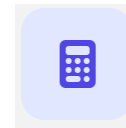


DEFINITION

단위 시간당 완료된 작업의 총 수

개발 팀이 특정 기간(주, 월 등) 동안 프로덕션에 배포하거나 완료(Done) 상태로 이동시킨 작업 항목의 개수를 의미합니다.

- ✓ 팀의 **생산성**을 나타내는 대표 지표
- ✓ 스토리 포인트 추정 없이 **객관적 측정** 가능
- ✓ 일정한 속도를 유지하는지 **안정성** 확인

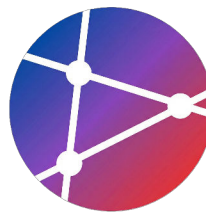


CALCULATION

$$\text{Throughput} = \text{Items} / \text{Time}$$

Example

1주일에 **10개**의 PR 완료
= 10 items/week



Throughput: 측정 예시

주간 처리량 자동화

GitHub Actions의 스케줄러를 활용하여 **주 단위 완료 작업 수**를 자동으로 집계하고 리포팅합니다.

작동 원리

- `schedule` 트리거: 정해진 시간(매주 일요일) 실행
- `gh issue list`: GitHub CLI로 이슈 목록 조회
- 닫힌(closed) 이슈만 필터링하여 개수 집계

 파일명: `.github/workflows/throughput.yml`

throughput.yml

```
# GitHub Actions로 주간 처리량 계산
name: Weekly Throughput
on:
  schedule:
    - cron: '0 0 * * 0' # 매주 일요일 자정 실행

jobs:
  calculate-throughput:
    runs-on: ubuntu-latest
    steps:
      - name: Get Closed Issues
        run: |
          # GitHub CLI를 사용하여 닫힌 이슈 조회 및 카운트
          CLOSED_ISSUES=$(gh issue list --state closed \
            --json closedAt --jq 'length')
```



SECTION 04

코드 품질 메트릭

Code Coverage, Technical Debt 등
소프트웨어의 내부 건전성을 평가하는 지표입니다.



Code Coverage



테스트 스위트 실행 시
소스 코드의 어느 부분이 실행되었는지를
나타내는 품질 지표

💡 왜 중요한가?

- ✓ 테스트 누락 영역 식별
- 🛡 소프트웨어 안정성 지표
- 🗑 불필요한 코드(Dead Code) 감지
- ✂ 리팩토링 안전망 제공

CALCULATION FORMULA

$$\text{Coverage} = \frac{\text{실행된 라인 수}}{\text{전체 라인 수}} \times 100\%$$

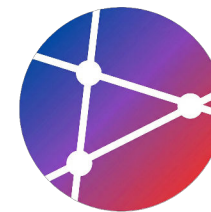
VISUAL EXECUTION

● Covered ● Uncovered



$$(6 / 8 \text{ lines}) \times 100 = 75\% \text{ Coverage}$$

🎯 Code Coverage 목표 수준



최소 수준

70%

프로젝트의 안정성을 보장하기 위한 최소한의 마지노선입니다. 이보다 낮으면 리스크가 큼니다.



권장 수준

80-90%

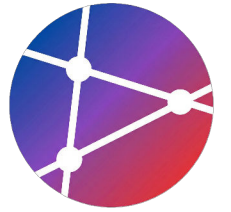
대부분의 버그를 사전에 차단하고 높은 유지보수성을 확보할 수 있는 가장 이상적인 구간입니다.



주의 사항

Wait!

100% 수치 달성에 집착하지 마세요. 의미 없는 테스트보다 **테스트의 품질**이 훨씬 중요합니다.



Code Coverage 자동화

GitHub Actions 예시

테스트 실행부터 커버리지 리포트 업로드, 그리고 목표 수치 달성 여부까지 자동으로 검증하는 파이프라인입니다.

📌 주요 단계

- **Run Tests:** `npm test --coverage`로 실행
- **Upload:** Codecov 액션을 사용하여 결과 저장
- **Check Threshold:** 커버리지가 80% 미만일 경우 빌드 실패 처리

📄 파일명: `.github/workflows/coverage.yml`

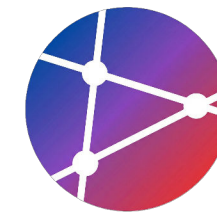
```
coverage.yml

# Code Coverage Check Workflow
name: Code Coverage
on: [push, pull_request]

jobs:
  coverage:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Tests with Coverage
        run: |
          npm test -- --coverage
      - name: Upload Coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/coverage-final.json
```



Technical Debt (기술 부채)



"코드 품질 문제로 인해 발생하는 미래의 추가적인 수정 비용을 의미합니다. 지금 쉬운 방법을 선택하면, 나중에 이자와 함께 갚아야 합니다."

- Ward Cunningham (1992)



초기 개발 속도는 빠르지만 장기적으로 느려짐



부채(Debt)처럼 시간이 지날수록 이자(비용)가 발생



결국 리팩토링(Refactoring)을 통해 상환해야 함

↔ 금융 부채 vs 기술 부채

개념

지금 당장 부족한 자원을 빌려 쓰는 것

원금

저품질 코드, 부족한 설계, 미비한 테스트

이자

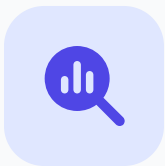
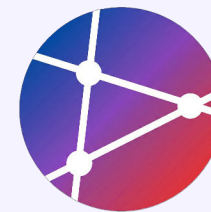
유지보수 난이도 증가, 버그 수정 시간 증가

상환

코드 리팩토링, 아키텍처 개선, 테스트 추가

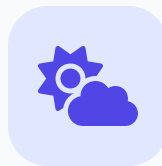


Technical Debt 측정 방법



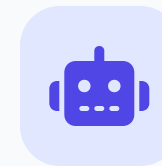
SonarQube

- ✓ **Code Smells:** 유지보수성 저해 요소
- ✓ **Bugs:** 신뢰성 문제 및 오류
- ✓ **Vulnerabilities:** 보안 취약점



Code Climate

- ✓ **GPA Score:** 코드 품질 등급 산정
- ✓ **Maintainability:** 유지보수 용이성 평가
- ✓ **Duplication:** 코드 중복도 분석



DeepSource

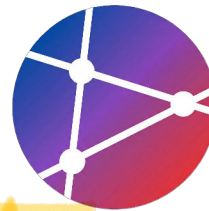
- ✓ **Auto Review:** 자동화된 코드 리뷰
- ✓ **Anti-patterns:** 안티 패턴 실시간 감지
- ✓ **Performance:** 성능 저하 요소 식별



Technical Debt: GitHub Actions

원래는 연동)

구독소스기여자



자동화된 코드 품질 관리

GitHub Actions와 SonarCloud를 연동하여 PR 단계에서부터 기술 부채를 식별하고 차단합니다.

⚙️ 주요 설정 포인트

- **fetch-depth: 0**: 정확한 분석을 위해 전체 Git 히스토리 필요
- **Secrets 관리**: SONAR_TOKEN 등 민감 정보 보호
- **Quality Gate**: 기준 미달 시 빌드를 실패시켜 배포 차단

📄 파일명: .github/workflows/code-quality.yml

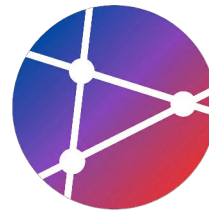
code-quality.yml

```
# .github/workflows/code-quality.yml
name: Code Quality
on: [push, pull_request]

jobs:
  sonarcloud:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          # SonarCloud 분석을 위한 전체 히스토리 가져오기
          fetch-depth: 0

      - name: SonarCloud Scan
        uses: SonarSource/sonarcloud-github-action@master
```

Code Churn (코드 변동률)



정의 (Definition)

특정 기간 동안 파일이 얼마나 자주 변경되었는지를 나타내는 지표입니다. 코드의 **추가(Added)**, **수정(Modified)**, **삭제(Deleted)**된 라인 수를 합산하여 계산하며, 개발 활동의 강도와 변화를 측정합니다.



의미와 해석

코드가 안정화되지 않고 계속 변경됨을 의미합니다. 출시 직전의 높은 변동률은 버그 발생 위험이 높다는 신호이며, 특정 모듈의 지속적인 높은 Churn은 **기술 부채**와 **리팩토링의 필요성**을 시사합니다.



High Code Churn

높은 코드 변동률



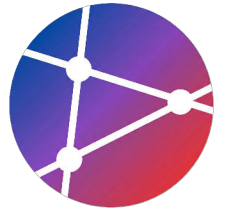
Unstable Code

코드 불안정성 증가



Needs Refactoring

리팩토링 1순위 대상



>_ Code Churn 측정 스크립트

Git 로그 분석 자동화

간단한 Bash 스크립트를 사용하여 지난 주 동안 변경된(추가/삭제) 코드 라인 수를 집계하고 **코드 변동률(Churn)**을 측정합니다.

🔧 핵심 명령어

- `git log --numstat`: 파일별 변경 라인 수 출력
- `awk`: 텍스트 데이터를 처리하여 합계 계산
- 추가된 줄(Added)과 삭제된 줄(Removed)의 총합이 Churn

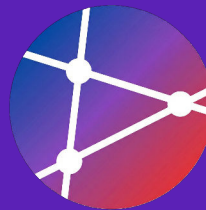
📄 파일명: `measure_churn.sh`

```
measure_churn.sh

#!/bin/bash
# 지난 주 동안의 Code Churn 계산

git log --since="1 week ago" --numstat --pretty=format:"" | \
awk '{
added += $1;
removed += $2
} END {
print "Lines Added:", added
print "Lines Removed:", removed
print "Total Churn:", added + removed
}'

# 실행 권한 부여: chmod +x measure_churn.sh
# 실행: ./measure_churn.sh
```



SECTION 06

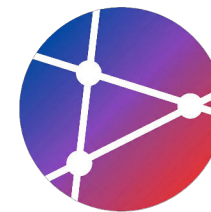
메트릭 대시보드 구축

데이터를 시각화하여 실시간으로 모니터링하고,
팀의 의사결정을 지원하는 대시보드를 구축합니다.





GitHub Insights 활용



Pulse (활동 요약)

프로젝트의 최근 활동 상태를 요약하여 보여줍니다. Merge된 PR 수, 해결된 Issue 수 등을 통해 프로젝트의 활성도를 한눈에 파악합니다.



Contributors (기여자 통계)

프로젝트 기여자들의 활동을 분석합니다. 커밋 빈도, 추가/삭제된 코드 라인 수 등을 통해 팀원별 기여도를 시각적으로 확인합니다.



Traffic (방문자 통계)

저장소의 방문자 수와 클론 횟수를 추적합니다. 유입 경로와 인기 있는 콘텐츠를 분석하여 프로젝트의 관심도를 측정합니다.

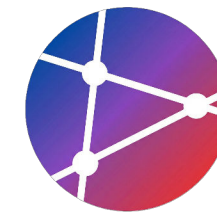


Commits (커밋 활동)

시간대별, 요일별 커밋 패턴을 분석합니다. 개발 팀의 주 활동 시간과 작업 리듬을 파악하여 협업 효율을 높이는 데 활용합니다.



커스텀 대시보드 개요



03로 시작하세요



Prometheus (데이터 저장)

Pull 방식의 메트릭 수집 및 시계열 데이터베이스(TSDB)를 통해 대용량 데이터를 효율적으로 저장하고 PromQL로 쿼리합니다.



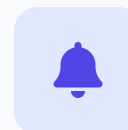
Grafana (시각화)

Prometheus 등 다양한 데이터 소스를 통합하여, 직관적인 그래프와 차트로 구성된 유연한 실시간 대시보드를 제공합니다.



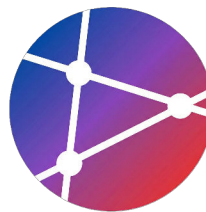
Exporters (데이터 수집)

OS, 데이터베이스, 애플리케이션 등 다양한 시스템의 상태를 Prometheus가 이해할 수 있는 포맷으로 노출합니다.



Alert Manager (알림)

설정된 임계값을 기반으로 이상 징후를 감지하고, Slack, Email 등 다양한 채널로 경고 알림을 발송합니다.



Grafana + Prometheus 예시

커스텀 대시보드 인프라

오픈소스 모니터링 표준인 **Prometheus**와 시각화 도구 **Grafana**를 Docker Compose로 손쉽게 구축할 수 있습니다.

구성 요소

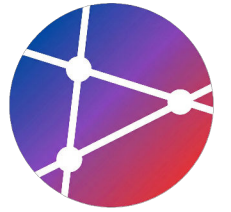
- **Prometheus (9090)**: 메트릭 수집 및 시계열 DB 저장
- **Grafana (3000)**: 수집된 데이터의 시각화 대시보드
- **Volumes**: 설정 파일 및 데이터 영구 저장소 연결

파일명: docker-compose.yml

docker-compose.yml

```
# docker-compose.yml
version: '3'
services:
  prometheus:
    image: prom/prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - 9090:9090

  grafana:
    image: grafana/grafana
    ports:
      - 3000:3000
    environment:
```



GitHub Actions로 메트릭 수집

데이터 수집 및 저장 자동화

정기적인 스케줄에 따라 GitHub API를 호출하여 프로젝트 메트릭을 수집하고, 이를 저장소에 기록하여 시계열 데이터를 구축합니다.

🔄 주요 프로세스

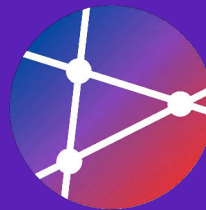
- **Schedule Trigger:** `cron` 을 사용하여 매일 자정에 자동 실행
- **Data Collection:** `gh api` 로 기여자 통계 등 데이터 추출
- **Persistence:** 결과를 JSON 파일로 저장 후 Git에 커밋/푸시

📄 파일명: `.github/workflows/collect-metrics.yml`

collect-metrics.yml

```
name: Collect Metrics
on:
  schedule:
    - cron: '0 0 * * *' # 매일 자정

jobs:
  collect:
    runs-on: ubuntu-latest
    steps:
      - name: Collect DORA Metrics
        run: |
          # GitHub API로 메트릭 수집
          gh api repos/${{ github.repository }}/stats/contributors \
            > metrics/contributors.json
```



SECTION 07

메트릭 기반 개선 프로세스

데이터에 기반한 의사결정과
지속적인 성과 향상을 위한 실행 사이클을 학습합니다.



1. 현상 측정 (Measure)

현재 상태를 정확히 파악하는 것이 개선의 첫 걸음입니다.

Lead Time for Changes



Current Value:

?

코드 커밋부터 배포까지의 시간

Deployment Frequency



Current Value:

?

프로덕션 배포 빈도

Mean Time to Recovery



Current Value:

?

장애 발생 시 복구 평균 시간

Change Failure Rate



Current Value:

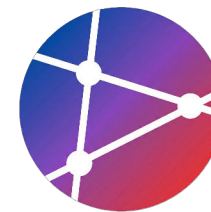
?

배포 실패 또는 롤백 비율



목표 설정

Set Goals



S

Specific

구체적

M

Measurable

측정 가능

A

Achievable

달성 가능

R

Relevant

관련성

T

Time-bound

기한

SMART Goal Example

CURRENT STATE (현재)

🕒 Lead Time: 5일

배포까지 평균 5일 소요

Improvement

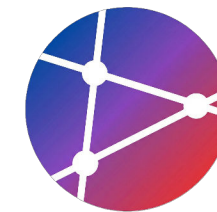


TARGET STATE (목표)

🚩 Lead Time: 1일

3개월 내 달성 (Time-bound)

실험 및 개선 (Experiment)



가설 수립

개선 아이디어 정의



실험 수행

변경 사항 적용



결과 측정

데이터 수집 및 분석



학습 및 반영

다음 액션 결정

실전 적용 예시: Lead Time 단축

가설

"자동화된 테스트를 파이프라인에 추가하면 수동 검증 시간이 줄어들어 Lead Time이 감소할 것이다."

측정

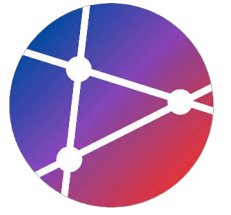
배포까지의 Lead Time 변화를 2주간 추적한다.
(예: 5일 → 3일로 단축 확인)

실험

주요 기능에 대한 단위 테스트를 작성하고 GitHub Actions CI 파이프라인에 통합한다.

학습

테스트 커버리지를 80%까지 확대하고, 통합 테스트(Integration Test) 단계도 자동화하기로 결정한다.



4. 지속적 모니터링 (Monitor)



1. Dashboard

실시간으로 핵심 메트릭(DORA, Flow)을 시각화하여 현재 상태를 한 눈에 파악합니다.

Grafana, Datadog, GitHub Insights



2. Alert

설정된 임계값을 초과하거나 이상 징후 발생 시 담당자에게 즉시 알림을 발송합니다.

Slack, Email, PagerDuty



3. Action

문제를 분석하고 신속하게 복구하거나 병목 지점을 최적화하는 조치를 취합니다.

Rollback, Hotfix, Scale-out

Continuous Improvement Cycle

AI Open Source Software



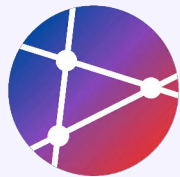
ASSIGNMENT

실습 과제

Practical Tasks

DORA Metrics 수집 자동화,
대시보드 구축 및 개선 계획 수립 실습을 진행합니다.





과제 1: DORA Metrics 수집 자동화

GitHub Actions를 활용하여 소프트웨어 전달 성능 지표를 자동으로 수집하고 리포팅하는 파이프라인을 구축합니다.

1 GitHub Actions 워크플로우 작성

Pull Request 및 Deployment 이벤트에 반응하는 .yaml 워크플로우 파일을 생성합니다.

2 4가지 핵심 지표 자동 계산

Lead Time, Deployment Frequency, MTTR, Change Failure Rate를 계산하는 로직을 구현합니다.

3 결과 데이터 저장

계산된 메트릭 데이터를 JSON 형식으로 Artifact에 저장하거나 별도 브랜치에 커밋합니다.

4 주간 보고서 자동 생성

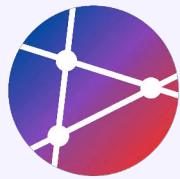
매주 수집된 데이터를 요약하여 Issue 또는 Slack 알림으로 전송하는 잡(Job)을 추가합니다.

제출 형식

- GitHub 리포지토리 URL 제출
- 워크플로우 파일 포함 (.github/workflows/metrics.yaml)
- 실행 성공 로그 스크린샷 1부
- README.md에 구현 방법 간략 기술

평가 포인트

- **자동화 완성도:** 수동 개입 없이 메트릭이 수집되는가?
- **정확성:** 지표 계산 로직이 DORA 정의에 부합하는가?
- **코드 품질:** 워크플로우 스크립트의 가독성 및 재사용성



과제 2: 메트릭 대시보드 구축

수집된 데이터를 시각화하여 팀의 소프트웨어 전달 성능을 실시간으로 파악할 수 있는 대시보드를 구축합니다.

1 GitHub API로 메트릭 수집

REST 또는 GraphQL API를 사용하여 필요한 메타데이터(이슈, PR, 커밋 등)를 주기적으로 가져옵니다.

2 시각화 도구 선택

Grafana, Chart.js, D3.js 등 프로젝트 규모와 팀의 기술 스택에 적합한 시각화 라이브러리를 선정합니다.

3 실시간 대시보드 구현

주요 지표(Cycle Time, 배포 빈도 등)를 차트 형태로 구성하여 웹 페이지나 모니터링 툴에 띄웁니다.

4 README 뱃지 추가

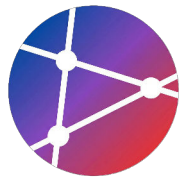
프로젝트 README.md 상단에 현재 상태(빌드 성공, 커버리지, 최신 배포일 등)를 보여주는 실시간 뱃지를 부착합니다.

제출 형식

- 대시보드 접속 URL 또는 스크린샷
- 구현 코드 (Frontend/Backend 리포지토리)
- 사용된 API 및 도구 명세서
- README에 부착된 뱃지 확인 링크

평가 포인트

- **가시성:** 데이터가 직관적으로 이해하기 쉬운가?
- **실시간성:** 지표가 최신 상태를 잘 반영하는가?
- **유용성:** 팀의 의사결정에 실질적 도움이 되는가?



과제 3: 개선 계획 수립

현재 프로젝트의 상태를 데이터 기반으로 분석하고, 병목 지점을 해결하기 위한 구체적인 개선 목표와 실행 계획을 수립합니다.

1

현재 프로젝트 메트릭 측정

GitHub Insights 및 구축한 대시보드를 활용하여 현재의 Lead Time, 배포 빈도 등 베이스라인 데이터를 수집합니다.

2

병목 지점 분석

측정된 데이터를 바탕으로 개발 프로세스 중 가장 시간이 많이 소요되거나 오류가 잦은 병목 구간을 식별합니다.

3

개선 목표 설정

분석 결과를 토대로 SMART 원칙(구체적, 측정가능, 달성가능, 관련성, 기한)에 입각한 개선 목표를 설정합니다.

4

실행 계획 작성

목표 달성을 위해 도입할 자동화 도구, 프로세스 변경, 테스트 전략 등을 포함한 구체적인 액션 플랜을 작성합니다.

제출 형식

- 개선 계획 보고서 (PDF 또는 Markdown)
- 현재 메트릭 상태 스크린샷 첨부
- 예상 성과 및 일정표(Roadmap)
- GitHub 저장소 내 IMPROVEMENT.md 파일로 제출

평가 포인트

- 데이터 기반 분석: 주관적 느낌이 아닌 측정 데이터에 근거했는가?
- 목표의 적절성: SMART 원칙을 준수하여 목표를 설정했는가?
- 실행 가능성: 구체적이고 현실적인 전략인가?



REFERENCES

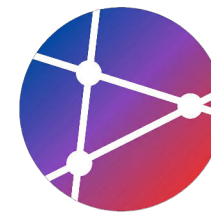
참고 자료 및 도구

소프트웨어 메트릭 학습에 도움이 되는
필수 도서, 도구, 추가 읽기 자료 모음입니다.





도구 (Tools)

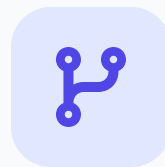


메트릭 수집

 Sleuth

 LinearB

 Haystack

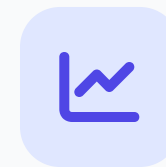


코드 품질

 SonarQube

 CodeClimate

 Codacy



시각화

 Grafana

 Kibana

 DataDog

SUMMARY



핵심 요약

강의의 주요 내용을 복습하고
실천을 위한 다음 단계를 확인합니다.



SUMMARY

Key Takeaways

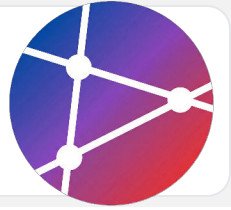
"측정할 수 없으면
관리할 수 없고,
관리할 수 없으면
개선할 수 없다."

- Peter Drucker

1

측정 없이는 개선 불가

현재 상태를 객관적으로 파악하는 것이 모든 개선의 시작점입니다.



2

DORA 4대 메트릭

Lead Time, Deployment Frequency, MTTR, Change Failure Rate를 기준으로 성과를 측정하세요.

3

Flow Metrics 활용

Cycle Time과 WIP 관리를 통해 개발 프로세스의 흐름과 효율성을 최적화할 수 있습니다.

4

자동화가 핵심

메트릭 수집부터 리포팅까지의 과정을 자동화하여 데이터의 정확성과 신속성을 확보하세요.

5

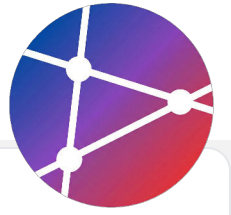
지속적 개선 사이클

측정(Measure) → 목표 설정(Goal) → 실험(Experiment) → 모니터링(Monitor)의 루프를 반복하세요.

Next Steps

"What gets measured gets managed."

- Peter Drucker



도구 설정 (Tool Setup)

프로젝트에 메트릭 수집 도구를 설정하여 데이터 확보 환경을 구축하세요.



기준선 설정 (Establish Baseline)

현재 상태를 측정하고 개선의 기준이 되는 베이스라인을 설정하세요.



목표 수립 및 실행 (Set Goals)

구체적인 개선 목표를 수립하고 이를 달성하기 위한 실행 계획을 수행하세요.



정기 리뷰 (Regular Review)

정기적인 메트릭 리뷰 회의를 통해 성과를 점검하고 방향을 조정하세요.



NEXT WEEK PREVIEW

다음 주차 학습 내용 예고

Week 2: CI/CD 파이프라인 구축

이번 주에 학습한 메트릭을 기반으로
실제 자동화된 배포 환경을 구성합니다.

